

Lumen - AI Skincare Assistant

Myat Bhone San, Ray Zhao, Shefali Saini, Sriyavarma Saripella

Master's in Software Engineering

San Jose State University, San Jose, United States

myatbhone.san@sjsu.edu, ray.zhao@sjsu.edu

shefali.saini@sjsu.edu, sriyavarma.saripella@sjsu.edu

Abstract—Lumen is an AI-powered skincare assistant designed to evaluate skin health, track changes over time, educate users, and provide personalized skincare insights. Lumen integrates machine learning models with conversational AI to deliver recommendations that are private, explainable, and accessible directly on iOS devices. The system employs local storage via SwiftData to ensure user privacy while optionally supporting AWS cloud-based analysis for scalability. Core features include AI-driven condition detection, progress tracking, and a guided camera interface built with SwiftUI. Lumen demonstrates how emerging generative and agentic AI technologies can enhance personal wellness through ethical and user-centric design.

Index Terms—Skincare, Productivity Tracking, Artificial Intelligence, Personalized Recommendations, Work-Life Balance, Stress Management, Cloud Computing, Backend Services, NoSQL, Authentication, Machine Learning, Data Analytics, HR Analytics, Survey Data, Workplace Productivity.

I. INTRODUCTION

Skincare is an integral part of personal wellness, yet access to professional dermatological advice remains limited. The Lumen project aims to bridge this gap by using artificial intelligence to provide an automated, privacy-conscious method for skin health assessment. The app allows users to capture facial images, analyze visible conditions such as acne, dryness, or pigmentation, and receive personalized recommendations within seconds.

Developed as an iOS application, Lumen leverages Swift 5.0, SwiftUI 3.0, and SwiftData to deliver a responsive and intuitive user interface. Its backend integrates Lambda, S3, and DynamoDB, Cognito, API Gateway for scalable AI-driven skin analysis.

II. SYSTEM ARCHITECTURE

Lumen employs a three tier architecture consisting of the iOS client, the AWS serverless backend, and a multi-model AI analysis layer. This architecture enables a fluid interaction between user input, image analysis, and AI-driven insight generation while maintaining strong security guarantees and low operational overhead.

A. Frontend (iOS Application)

The mobile application is built using Swift 5.0 and SwiftUI 3.0, which together provide a modern interface for user interactions. The camera interface is implemented using AVFoundation to ensure precise face alignment and consistent image quality across analyses. SwiftData manages local storage of user onboarding information and analysis history, enabling

offline browsing and immediate access to previously generated results with repeated network calls.

The frontend handles the entire user workflow, from onboarding to image capture to the presentation of analysis browsing. It also provides access to the Learn tab which includes AI chat interactions, article browsing, and personalized recommendations. All network interactions are authenticated through AWS Cognito, ensuring that each API request carries the appropriate JWT token for secure backend access.

B. Backend (AWS Cloud Infrastructure)

The backend is deployed as a serverless architecture using AWS Lambda and API Gateway. This design allows the system to scale automatically based on demand while minimizing operational complexity. API Gateway exposes REST endpoints for operations such as initiating an analysis, retrieving analysis history, generating insights, handling chat messages and serving Learn tab content.

When a user requests an analysis, the backend generates an analysis_id and returns a presigned URL that enables temporary, permission controlled upload to Amazon S3. Once the image is uploaded, a Lambda function retrieves the file from S3, performs model inference, computes a set of skin health metrics, and stores the results in a DynamoDB table keyed by user ID and timestamp. Additional Lambdas orchestrate product recommendations, and RAG-based article retrieval.

User authentication is handled entirely by AWS Cognito. Access to S3, DynamoDB, and Bedrock is controlled by IAM roles that follow the Principle of Least Privilege, ensuring each component receives only the permissions required to perform its designated tasks.

C. AI and Analysis Layer

The AI and Analysis layer integrates multiple specialized components.

Skin issue classification is handled through a deployed model running on a Hugging Face space, the model used in development was referenced from a space by Pratik and later we trained our models iteratively using a variety of datasets. The working implementation is currently achieved by allowing its inference endpoint to be called through the Lambda function and the raw image uploaded from the user is sent via HTTP post as multipart form data, the hugging face space essentially acts as an ECR repository with a Docker container hosting the model. The predictions with condition

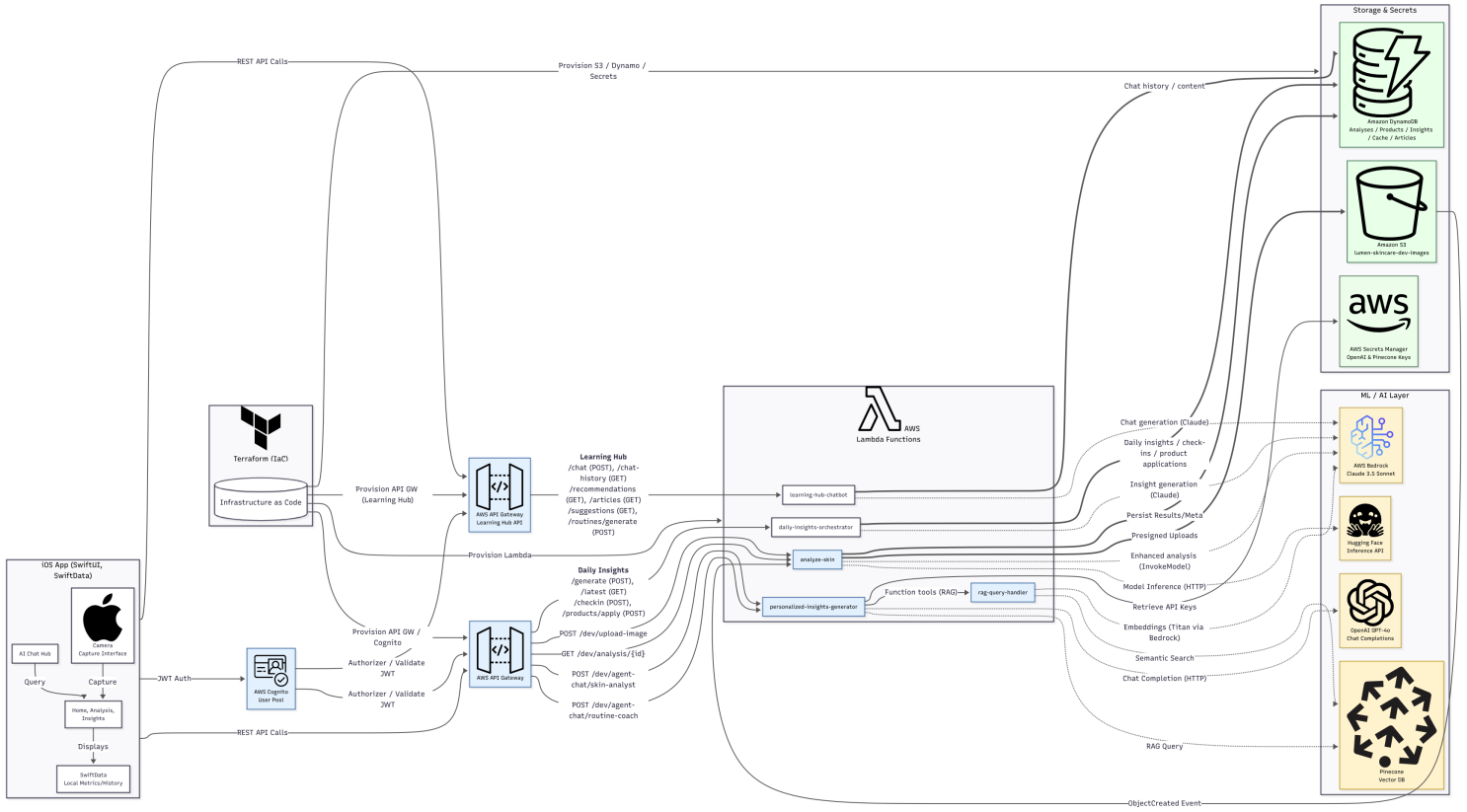


Fig. 1. System Architecture Diagram

prediction as well as confidence scores will then be returned in JSON format where the main Lambda handler function will start enhancing the analysis response with AWS Bedrock.

For generative reasoning, Lumen uses Claude 3.5 Sonnet through AWS Bedrock to produce user-friendly interpretations of the analysis results. The Hugging Face prediction is further enhanced with RAG to provide a detailed summary, recommendations, and severity with an external knowledge base on Pinecone DB.

OpenAI's GPT-4o model is incorporated into the insights component which is in the Home screen of Lumen. A Lambda function handles the flow of the AI agents Skin Analyst and Routine Coach after it receives the prediction results from Hugging Face and the Skin Analyst provides advice at the top section while the Routine Coach acts as an agent that provides motivational words of wisdom at the bottom. The agents will give new responses after each analysis while enhancing their output with the same external knowledge base from Pinecone.

III. SYSTEM WORKFLOW

A. Application Launch & Authentication

When the user opens the app, it automatically authenticates them through AWS Cognito. Once authenticated, the app receives a secure JWT token, which is used to authorize all subsequent requests to the backend.

B. Skin Analysis Initiation

The user selects the option to perform a skin analysis. The camera interface is opened, and the user captures a photo of their skin. After the photo is taken, the app sends a request to the backend's image-upload endpoint, passing along the authentication token.

C. Upload Preparation & Image Transfer

The backend responds with a newly generated analysis_id along with a presigned S3 upload URL. The app then uploads the captured image directly to the S3 bucket using this URL.

D. Analysis Status Monitoring

After the upload is complete, the app begins checking the status of the analysis by calling the /analysis/{analysis_id} endpoint. This continues until the backend finishes processing the image.

E. Image Processing & Metric Generation

Once the image is in S3, a backend Lambda function retrieves it and performs machine-learning inference using a skin-analysis model. The system computes multiple skin-health metrics—such as acne levels, dryness, pigmentation, texture, and overall skin health score. These results are saved in DynamoDB, and the analysis is marked as complete.

F. Results Delivery to Client

The app fetches the final analysis results from the backend. These metrics are displayed to the user in the analysis detail screen and are also stored locally via SwiftData for history tracking and offline access.

G. Historical Tracking

Users can review earlier analyses within the History section. Each analysis is stored as a SkinMetric record, allowing the user to view past results and observe trends over time.

H. AI Consultation via Agents

When the user seeks deeper insights or recommendations, they can interact with AI agents such as the Skin Analyst or Routine Coach. The app sends the user's message and the relevant analysis_id to the backend's agent-chat endpoint.

I. Backend AI Reasoning & Response Generation

The backend retrieves the user's latest analysis, gathers any relevant past context, and optionally performs a knowledge-base lookup using RAG (Retrieval-Augmented Generation). It then leverages GPT-4o to generate an informed, personalized response. This message is stored and returned to the client.

J. Chat Delivery & Local Persistence

The app receives the agent's response, displays it in the chat interface, and saves it locally as a ChatMessage entry, ensuring continuity across sessions.

K. Long-Term Insight & Progress Monitoring

As the user continues performing analyses over time, the system compares previous SkinMetric entries using built-in progress-tracking services. Daily insights or trends may be generated to motivate the user and help them understand changes in their skin health.

IV. DATABASE DESIGN

The **Analyses Table** stores detailed records of each skin evaluation performed by the AI model. It includes essential fields such as the user identifier, image reference, predicted skin condition, and timestamps to track the completion of the analyzes. By maintaining this information, the system enables users to review their skin progress over time and allows the

analysis_id	user_id	completed_at	Prediction (full response will be in JSON)	s3_key	status	timestamp	tll
886087ee-854f-4aad...	anonymous 1	1762332362	EYE BAGS	uploads/anonymous886087ee-...-jpg	completed	2025-11-05T08:46:02.81...	1770108361
8703753f-9ba1-f473a...	anonymous 2	1762807488	FOREHEAD WRINKLES	uploads/anonymous8703753f-...-jpg	completed	2025-11-10T20:44:48.96...	1770583485
58250346-239b-4e19...	anonymous 3	1762807511	HORMONAL ACNE	uploads/anonymous58250346-...-jpg	completed	2025-11-10T20:45:11.70...	1770583509

Fig. 2. Analyses Table

backend to monitor model performance. Expired or outdated entries are automatically removed using DynamoDB's time-to-live (TTL) feature, keeping the database clean and efficient.

The **Products Table** contains comprehensive information about the recommended skincare products for users. Each record includes product identifiers, names, categories, ingredients, and usage suitability for various skin types. This table supports the personalization engine by linking AI-generated recommendations with appropriate products, ensuring that users receive guidance that is aligned with their specific needs. The structure also allows for version control, making it easier to update or replace products when formulations change.

The **Chat History Table** maintains records of all user interactions with the Lumen assistant. Capture messages, timestamps, and sender information to provide context for ongoing conversations. This design helps create a continuous and natural dialog experience by allowing the assistant to refer to previous discussions. Data in this table are encrypted to ensure privacy, while pagination methods enable efficient retrieval during longer chat sessions.

The **Educational Content Table** holds learning materials such as articles, tutorials, and short videos on skincare-related topics. Each entry is organized by category and tagged for easy indexing, enabling the app to deliver personalized educational recommendations. The table supports regular content updates, allowing administrators to refresh and expand the knowledge base while maintaining accuracy and reliability.

The **Check-in Responses Table** collects self-reported feedback from users during their daily or weekly check-ins. Information such as mood, product usage, and visible changes in skin condition is stored here. These responses are compared with AI analysis data to provide a more holistic understanding of the user's skin journey. The structure of this table is intentionally flexible, allowing future updates or additional metrics to be integrated seamlessly.

The **Product Applications Table** tracks how and when users apply specific skincare products recommended by Lumen. By correlating product usage patterns with analysis outcomes, the system can evaluate the effectiveness of certain routines or ingredients. This information supports adaptive recommendations, ensuring users receive timely guidance on how to adjust their skincare habits for better results.

The **User Preferences Table** manages individual settings and personalization data, including app themes, notification preferences, and analysis frequency. It ensures that users have a tailored experience aligned with their comfort and privacy choices. The table also securely stores consent settings, enabling transparency and compliance with data protection standards.

Finally, the **System Logs Table** is used to monitor backend operations, including AI processing events, API responses, and performance metrics. It plays a crucial role in maintaining the reliability and stability of the application. By analyzing log data, developers can identify errors, optimize performance, and ensure that system operations run smoothly. Logs are

periodically archived or deleted to maintain efficiency and comply with privacy protocols.

V. FUNCTIONALITIES

A. Skin Analysis

The system provides an AI-powered skin analysis feature that evaluates user submitted or camera captured selfies. Using a guided camera interface and face alignment prompts, users are instructed to position their face correctly to ensure consistent image quality. The image is processed through a two-stage AI pipeline which identifies possible skin conditions such as acne, dryness, oiliness, wrinkles, dark spots or redness. The model produces both a condition classification and confidence scores, enabling accurate insights into the user's current skin health.

B. Onboarding Experience

The onboarding flow collects essential user details to support personalized skincare recommendations. These include:

- Name, Age, Height, Weight
- Skin concerns (e.g. acne, dryness, scarring, etc.)
- Primary skincare goal (e.g. clear skin, brightness, anti-aging etc.)

This information is securely stored and incorporated into the recommendation logic and AI chat responses. The onboarding process ensures that predictions and routine suggestions remain tailored to the specific user.

C. Personalized Recommendations

Based on model predictions and user profile data, Lumen suggests skincare routines, daily habits and verified Amazon products.

D. Educational Content

The Learning Hub provides users with easy-to-use features to help them learn more about skincare:

1) **AI Chat Assistant:** The AI chat allows users to ask questions about skincare routines, concerns, products and best practices. Responses are personalized based on the user and give simple and clear guidance whenever needed.

2) **Articles Library:** Users can browse a collection of educational skincare articles through categorized links. The articles cover topics such as acne care, hydration, sun protection and product tips.

3) **For You Page:** After completing the first skin analysis, the For You page will provide personalized article recommendations.

VI. PERSONA

A. New Skincare Learners

New skincare learners are beginners who often feel overwhelmed by the abundance of skincare trends, influencer advice, and contradictory product information available online. They may not understand ingredient labels, proper application methods, or how to identify their skin type.

Lumen simplifies this process by offering a guided onboarding experience that uses AI to analyze the user's skin through a photo and immediately provide a beginner-friendly routine with clear, minimal steps. The app's conversational assistant explains skincare terms in plain language, ensuring accessibility for users with no prior knowledge.

B. Everyday Skincare Users

Everyday skincare users are individuals who already follow a basic skincare routine but seek to improve it with more structure and understanding. They may experience occasional breakouts, dryness, or sensitivity but are unsure about the root causes or best remedies. Their primary goal is to maintain healthy skin while saving time and effort.

Lumen supports this group with routine tracking, progress visualization, and AI-driven recommendations that adapt to their changing skin conditions. The Routine Coach agent provides motivational support, suggesting simple improvements like consistency in moisturizing or adjusting product frequency. The Check-in feature encourages users to log daily feedback, while AI insights highlight correlations such as "Increased hydration observed after consistent moisturizer use."

C. Skincare Enthusiasts

Skincare enthusiasts already have a solid understanding of skincare science and want more control and data-driven insight into their skin's behavior. They are curious about ingredients, product comparisons, and quantifiable progress. Many of them track routine adjustments or keep before-and-after photos to evaluate long-term improvements.

Lumen caters to enthusiasts by offering in-depth analysis metrics such as hydration level, pigmentation patterns, wrinkle detection, and estimated skin age. These users can explore the Learning Hub, which provides research-based educational content powered by RAG (Retrieval-Augmented Generation) technology. The AI Skin Analyst generates detailed explanations, while product data stored in DynamoDB allows for ingredient-based recommendations and comparisons.

VII. TECHNOLOGIES USED

The development of Lumen – AI Skincare Assistant utilized a range of modern technologies and tools across mobile development, artificial intelligence, and cloud infrastructure. Each component was carefully chosen to ensure performance, scalability, privacy, and ease of use.

A. Frontend

The frontend of Lumen was developed as a native iOS application using Swift and SwiftUI frameworks. The major tools and frameworks include: Swift 5.0: Programming language used for implementing all application logic and UI components.

SwiftUI 3.0: Declarative user interface framework used to design the app's screens, including skin analysis, progress tracking, and skincare routine recommendations.

SwiftData: Used for local data storage and management of skincare analysis history, providing secure on-device persistence.

AVFoundation: Integrated for camera functionality to enable real-time facial photo capture with a guided interface.

PhotosUI: Provides access to the device's photo library for selecting existing images for analysis.

Xcode 16.1: The Integrated Development Environment (IDE) used for coding, debugging, and running the app on iPhone simulators.

B. Backend and Cloud Technologies

The backend and cloud architecture were deployed using AWS services to support AI-driven analysis and data processing. The major cloud technologies used are:

AWS Lambda (Python 3.11): Handles serverless computation for image processing and AI inference, all API gateway endpoints are handled through Lambda as well.

Amazon S3: Provides secure image storage through pre-signed URLs to ensure temporary and private uploads.

Amazon DynamoDB: Used for storing and managing analysis results and user progress metrics, product items with valid Amazon links, articles with reputable links for the Learning Hub section integration.

Amazon API Gateway: Acts as an interface between the mobile app and backend services.

AWS Cognito: Manages user authentication and access control using JWT-based security. An added layer of security for API authentication when calls are made to and from the client side from the iOS app.

Terraform: Used for automating infrastructure deployment and managing AWS resources.

Pinecone DB: The main knowledge base accessed with RAG for evidence-based recommendations

C. Artificial Intelligence and Machine Learning Technologies

Lumen integrates machine learning and generative AI models to detect skin issues and generate personalized insights. The AI components include:

Claude 3.5 Sonnet (AWS Bedrock): A large language model used for generating recommendations, explanations, and conversational analysis results. We are using the Bedrock Runtime ID in order to invoke the model and use it with tool calling and access to DynamoDB tables with chat history and analysis tables with RAG to a vector knowledge base. This integrates seamlessly into the backend.

OpenAI GPT-4o: The primary model used for the agents in personalized insights, Skins Analyst and Routine Coach.

Python 3.11: Used for developing and deploying the Lambda-based AI inference logic.

EfficientNet-B0: A deep learning model fine-tuned for identifying skin issues such as acne, dryness, and pigmentation from user images.

D. Tools and Development Environment

macOS Sequoia 15.5 / Tahoe 26.1: The operating systems used for developing and testing the iOS application.

Xcode Simulator (iPhone 16/17): Used for testing and debugging the application during development.

Git and GitHub: Version control and collaboration tools used for maintaining the project source code.

Postman / curl commands: Utilized for testing API endpoints and validating AWS Lambda responses.

E. Security and Privacy Technologies

JSON Web Tokens (JWT): Used for secure authentication between the app and AWS backend via Cognito.

HTTPS Protocol: Ensures encrypted communication for all network requests.

Local Data Encryption (SwiftData): Protects user analysis history stored on-device, ensuring privacy and compliance with ethical AI principles.

VIII. TESTING

A series of tests were conducted to evaluate the stability, correctness, and reliability of Lumen across its major features and workflows. The testing process combined manual validation using the iOS simulator with automated checks to ensure that core functionality, data flow, and user-facing interactions behaved as expected. These evaluations confirmed that the application performed consistently under typical usage conditions.

A. UI Testing

UI testing focused on verifying that each user-facing workflow rendered correctly, responded to user actions, and maintained consistent behavior across screens. Automated XCTest tests were used extensively to validate all major user flows, ensuring stability, reliability, and demo-ready behavior across the app.

1) Onboarding Flow:

- Validates required inputs, including name, skin concerns, skincare goals
- Verifies optional body metric entry functionality
- Confirms full onboarding progression
- Detects returning user and correctly bypasses onboarding on relaunch

2) Home Screen Validation:

- Confirms that the Home screen loads successfully
- Checks visibility and accessibility of all tiles (New Scan, History, AI Chat, Learn)
- Tests scroll behavior and header rendering
- Verifies correct behavior of tab navigation

3) Camera Flow Testing:

- Opens the camera interface from Home screen
- Ensures camera UI loads consistently
- Handles system permission prompts when displayed
- Confirms that the close button returns to the Home screen
- Verifies presence and responsiveness of the shutter button

4) *History Tab Verification:*

- Opens the History tab from both the tile and tab bar
- Validated correct rendering of list and empty states
- Opens a history detail entry when entries are available

5) *Learn Tab & AI Assistant Testing:*

- Opens the Learn tab from both the tab bar and Home tile
- Loads default Chat tab and verifies switching to *For You* and *Articles*
- Initiates an AI chat conversation successfully
- Opens an article from Articles tab

B. *Performance Testing*

Performance testing assessed the responsiveness and latency across both real and simulated devices, including physical iPhone 11 hardware and iPhone 16 and iPhone 17 Pro simulators. Average skin analysis time ranged from 3 to 4 seconds, reflecting efficient backend computation and serverless execution concurrency. Application startup averaged 1.2 seconds, demonstrating optimized initial loading pathways. Throughout testing, the interface remained fluid with minimum latency, confirming that SwiftUI components rendered efficiently and backend endpoints returned stable response times.

C. *API, Security, and Privacy Testing*

API, security, and privacy testing focused on verifying that the system handled authenticated requests correctly and protected user data throughout the analysis workflow. Basic endpoint validation was performed using the iOS application and Postman to ensure that all API calls required valid JWT tokens issued by AWS Cognito and that unauthorized requests were rejected as expected. Presigned S3 upload URLs were confirmed to function correctly within their expiration window, preventing upload outside of the allowed timeframe.

The application's data flow was examined to confirm that all communication between the client and backend occurred over HTTPS and that image uploads, analysis results, and user profile information were transmitted securely. DynamoDB and S3 storage behavior was validated to ensure that data remained accessible only through the appropriate Lambda functions and not through unintended pathways. Local data stored via SwiftData also operated as expected, remaining confined to the device and properly encrypted by iOS.

Overall, API interaction, authentication mechanisms, and data-handling behavior exhibited no issues during testing. The system consistently adhered to its security model, and all tested components functioned reliably under normal usage conditions.

Fig. 3. Frontend testing code coverage

Fig. 4. Backend testing code coverage

IX. API SECURITY

The Lumen AI Skincare Assistant backend has been designed with strong security measures to ensure that all user interactions, images, and AI analysis results remain confidential and protected. The system follows a layered security model that combines authentication, authorization, encryption, and continuous monitoring using AWS cloud services.

A. *Authentication and Authorization*

User authentication in Lumen is implemented through AWS Cognito, which manages user identities and ensures secure access to all backend resources. Each user session is verified using JWT (JSON Web Tokens), which are issued upon login and validated by AWS API Gateway before any request reaches the backend services. These tokens are short-lived and automatically refreshed, minimizing the risk of unauthorized reuse or session hijacking. Every API request is linked to a unique user identity, ensuring strict data isolation and preventing users from accessing another individual's data. For development and testing purposes, a secure demo user pool is configured, allowing limited access to non-sensitive endpoints while maintaining full security enforcement.

B. *Secure Communication*

All communication between the iOS application and the backend infrastructure is fully encrypted using HTTPS with TLS 1.2 or higher, ensuring the confidentiality and integrity of transmitted data. The system uses presigned URLs to manage photo uploads securely, allowing temporary, permission-controlled access to Amazon S3 storage. This approach ensures that only authorized uploads occur within a short validity period. Data stored in both Amazon S3 and Amazon DynamoDB is encrypted at rest using the AES-256 encryption standard, meaning even internal users must have appropriate authorization to access stored data. Additionally, temporary image files created during skin analysis are automatically deleted once the process is complete, minimizing the risk of exposure or unauthorized reuse.

C. *API Gateway and Access Control*

AWS API Gateway serves as the central, secure entry point for all client-side requests. It validates each incoming JWT token, filters malformed requests, and blocks unauthorized traffic before invoking any backend service. Communication between backend components—such as AWS Lambda, S3, and DynamoDB—is regulated using IAM roles and policies that follow the Principle of Least Privilege, ensuring that each service only has the minimum permissions required to perform its function. To protect against misuse, rate limiting and request throttling are enforced at the gateway level to prevent excessive traffic or potential denial-of-service (DoS) attacks. System logs and metrics are continuously captured by AWS CloudWatch, which provides real-time monitoring of authentication attempts, network anomalies, and security exceptions, enabling proactive threat detection and response.

D. AI Model Security

The AI inference layer within Lumen is powered by AWS Bedrock, specifically the Claude 3.5 Sonnet model, which operates inside a private VPC endpoint to prevent direct internet exposure. All inference requests are authenticated using Cognito-issued credentials, ensuring that only authorized users or system components can initiate AI-based predictions. The input data sent to Bedrock, including user images and associated metadata, remains confined within the Lumen environment and is never stored or shared externally. Furthermore, Bedrock's infrastructure adheres to leading compliance standards such as SOC 2, GDPR, and ISO 27001, which guarantees secure, ethical, and compliant handling of AI workloads throughout the analysis process.

E. Data Privacy and Governance

Lumen follows a strict privacy-by-design framework where no user data is shared with any external party. All data remains securely stored within the AWS infrastructure managed solely by the project team. CloudWatch logs are sanitized to remove user-identifiable information, ensuring that monitoring activities remain anonymous. User images and analytical reports are referenced through hashed identifiers, maintaining privacy even within internal systems. On the client side, SwiftData handles local data storage in an encrypted format, which is further protected by Apple's Secure Enclave technology. This ensures that sensitive information such as user preferences, chat history, and analysis results remain fully private, accessible only within the authorized device and application context.

X. FUTURE ENHANCEMENTS

- Implement user login functionality to provide a better personalized experience.
- Collect and incorporate user feedback to improve application features and its usability.
- To enhance observability for the developers by integrating CloudWatch metrics on the homepage, enabling monitoring of app performance and its usage patterns. This includes tracking analysis metrics, user feedback, and visualizing thumbs-up/thumbs-down interactions as time-series data.

XI. CONCLUSION

The Lumen – AI Skincare Assistant successfully demonstrates how AI and cloud services can be combined to build a personalized and privacy-first skincare application. The project meets its objectives by enabling users to analyze skin health, track progress, and receive AI-driven skincare advice.

Built using SwiftUI, SwiftData, and AWS Bedrock (Claude 3.5 Sonnet), the system achieves efficient performance while maintaining full data security. Its modular architecture allows offline operation and scalable cloud integration.

Future improvements include integrating Apple HealthKit, deploying Core ML models for on-device AI, and adding multilingual support. Overall, Lumen highlights how agentic AI can enhance user wellness through intelligent, secure, and user-centered design.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Rakesh Ranjan for providing valuable guidance, continuous support, and encouragement throughout the development of our project, Lumen – AI Skincare Assistant.

Special thanks to Amazon Web Services (AWS) for access to Bedrock and related cloud tools, which allowed us to integrate generative AI capabilities such as Claude 3.5 Sonnet into our system.

Lastly, we would like to acknowledge the collective effort of our teammates Myat Bhone San, Ray Zhao, Shefali Saini, and Sriyavarma Saripella for their dedication, collaboration, and creativity in every stage of development, from design to implementation and testing.

REFERENCES

For Hugging Face Model Training and Inferences

We would like to give thanks to Pratik Smitin Karnik as we referenced his model, and got the idea for hosting the inference through Hugging Face spaces.

https://huggingface.co/spaces/pratikskarnik/face_problems_analyzer/tree/main

For Dataset Collection and Preparation

We would like to extend our gratitude to Ahmad Ismail, the creator of the Skin Issues Version 2 – Balanced Dataset on Kaggle. This dataset provided a diverse and well-structured foundation for training and evaluating our AI skincare detection model.

<https://www.kaggle.com/datasets/ahmedismaail/skin-issues-version-2-dataset-balanced>

For Dataset Annotation and Expansion

We would like to acknowledge the contribution of the Roboflow community for providing the Skincare Dataset on the Roboflow Universe platform. This dataset was instrumental in enhancing our model's accuracy by offering additional annotated images for various skin conditions.

Roboflow dataset: <https://universe.roboflow.com/skincare-gxqjb/skincare-8dmzr/dataset/7>